Secure Software Design and Development (Part II: Seven Touchpoints for Software Security)

CHAPTER 5. INTRODUCTION TO SOFTWARE SECURITY TOUCHPOINTS

BOOK: SOFTWARE SECURITY: BUILDING SECURITY IN, 1ST EDITION BY GARY MCGRAW

Seven Touchpoints

Seven Touchpoints [1/2]

1. Code Review (Tools)
2. Architectural Risk Analysis
3. Penetration Testing
4. Risk-Based Security Testing
5. Abuse Cases
6. Security Requirements
7. Security Operations

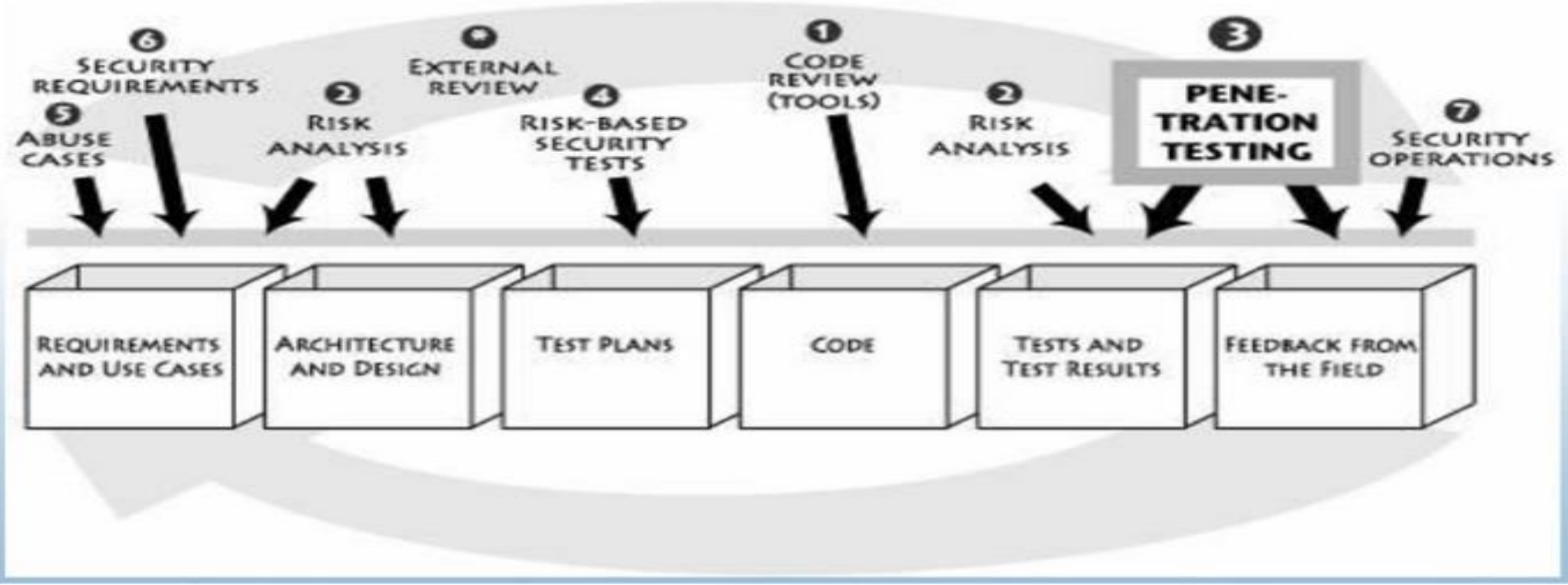
Penetration Testing

Penetration Testing: Introduction [1/2]

- Software penetration testing is the practice of testing a computer system, network or web application to find vulnerabilities that an attacker could exploit

Penetration Testing

Penetration Testing: Introduction [1/2]



Penetration Testing

Penetration Testing Usual Approach

- Most commonly used today
- Currently
  - Outside->in approach
  - Better to do after code review and ARA
  - As part of final preparation acceptance regiment
  - One major limitation
    - Almost always a too-little-too-late attempt at the end of a development cycle

Penetration Testing

Penetration Testing Problem [1/2]

- Security testing needs to make use of both white hat and black hat concepts and approaches
- Most all penetration testing relies on black hat methods
  - Thinking like a bad guy
- Difficult to prove if system is secure enough under attack

Penetration Testing

Penetration Testing Problem [2/2]

- Penetration testing only identifies a small sample of all possible security risks
- Focus on a small list of issues will only mitigate a subset of security risks
- Greater security risks may still be in the system

Penetration Testing

Penetration Testing: A Better Approach

- Testing should be an approach based testing
  - Use the findings discovered and tracked at the beginning of the SDLC
- Penetration testing is about testing a system in its final production environment
- Drive test that concentrate on these factors with some knowledge of risk analysis
  - most effective approach

Penetration Testing

Penetration Testing: Use Tools

- Tools should be used for penetration testing
  - Static analysis tool – vet software code
  - Dynamic analysis tool – observer system as it executes
- Faults are reported back to tester for analysis
- Tools should be guided by risk analysis and attack patterns

Penetration Testing

Penetration Testing: Tools Benefits

- Two major benefits:
  - Tools carry about the majority of the grunt work
    - Shouldn't replace review of skilled security analyst
  - Tools output lends itself readily for metrics
    - Teams can use metrics to track progress over time
    - Clean bill of heath from analysis tool does not mean that a system is defect free

Penetration Testing

Penetration Testing: Test More Than Once

- Automated review is best suited for identifying the most basic of implementation defects
- Human review is necessary to reveal flaws in the design or complex implementation level vulnerabilities
  - Expert review

Penetration Testing

Incorporating Findings into Development

- Results of penetration test should propagate back into the organization
- Mitigation strategy is a critical aspect of any penetration test
  - Root cause analysis
- Use test result information to measure progress against a goal

Penetration Testing

Using Penetration Tests to Assess the Application Landscape

- Penetration test can help with software already created
  - Create basic risk analysis to pin down business context
  - Run uniform, fixed-length, standardized penetration test
  - Rank them according to their results

Penetration Testing

Penetration Testing Summary

- Penetration test should be driven by perceived risk
- Best suited for finding configuration problems and other environmental factors
- Make use of tools
  - Takes care of majority of grunt work